

Algoritmo de Wang — Demostrador de Teoremas v1

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Fundamento Teórico	2
2.1. Secuentes	2
2.2. Axioma de cierre	2
3. Reglas de Inferencia	2
4. Implementación del Algoritmo	2
5. Parser de Fórmulas	3
6. Interfaz Gráfica	3

1. Descripción del Proyecto

Primera versión del demostrador de teoremas basado en el **Algoritmo de Wang** (1960). Determina automáticamente si un argumento lógico proposicional es una tautología usando el cálculo de secuentes.

2. Fundamento Teórico

2.1. Secuentes

Un **secuente** tiene la forma $\Gamma \vdash \Delta$ donde:

- Γ (antecedente): conjunto de premisas
- Δ (consecuente): conjunto de conclusiones

El secuente es **válido** si, cuando todas las fórmulas de Γ son verdaderas, al menos una de Δ también lo es.

2.2. Axioma de cierre

Una rama se **cierra** (es válida) cuando la misma fórmula aparece en el antecedente y en el consecuente: $A, \Gamma \vdash \Delta, A$.

3. Reglas de Inferencia

Regla	Nombre	Transformación
$\neg L$	Negación-izq.	$\neg A, \Gamma \vdash \Delta \Rightarrow \Gamma \vdash A, \Delta$
$\neg R$	Negación-der.	$\Gamma \vdash \neg A, \Delta \Rightarrow A, \Gamma \vdash \Delta$
$\wedge L$	Conjunción-izq.	$A \wedge B, \Gamma \vdash \Delta \Rightarrow A, B, \Gamma \vdash \Delta$
$\rightarrow L$	Implicación-izq.	$A \rightarrow B, \Gamma \vdash \Delta \Rightarrow \neg A, B, \Gamma \vdash \Delta$
$\rightarrow R$	Implicación-der.	$\Gamma \vdash A \rightarrow B \Rightarrow A, \Gamma \vdash B$

4. Implementación del Algoritmo

```
1 boolean esValido(List<String> antecedente,
2                   List<String> consecuente) {
3     // Caso base: axioma de cierre
4     for (String f : antecedente)
5       if (consecuente.contains(f)) return true;
6
7     // Aplicar reglas al antecedente
8     for (String f : antecedente) {
9       if (f.startsWith("!")) { // negacion izq
10        List<String> nuevoAnt = sin(antecedente, f);
11        List<String> nuevoCons = con(consecuente,
12                                     f.substring(1));
13        return esValido(nuevoAnt, nuevoCons);
14      }
15      if (f.contains("->")) { // implicacion izq
```

```

16         String[] partes = f.split("->");
17         List<String> nuevoAnt = sin(antecedente, f);
18         nuevoAnt.add("!" + partes[0]);
19         nuevoAnt.add(partes[1]);
20         return esValido(nuevoAnt, consecuente);
21     }
22     // ... m s reglas
23 }
24 return false; // no se pudo cerrar
25 }

```

5. Parser de Fórmulas

El parser convierte la entrada del usuario en un árbol sintáctico:

- $\neg A \rightarrow$ negación $\neg A$
- $A \& B \rightarrow$ conjunción $A \wedge B$
- $A | B \rightarrow$ disyunción $A \vee B$
- $A \rightarrow B \rightarrow$ implicación $A \rightarrow B$

6. Interfaz Gráfica

- Campos de texto para ingresar premisas y conclusión.
- Panel de salida que muestra cada paso de la demostración.
- Resultado final: **VÁLIDO** (tautología) o **NO VÁLIDO** (refutable).